

SS-1201 Lab 02 (Part 3/3): Loops & Methods (Void & Value)

Objectives:

1. Learn Java repetition statements by covering the following topics:

Completed?	Sub-topics
Yes / No	while loops
Yes / No	do-while loops
Yes / No	for-loops
Yes / No	Nested loops
Yes / No	Nested if statements
Yes / No	switch statement with default option
Yes/ No	switch statement with no default option

2. Void Methods

3. Value Methods

4. Complete all examples, practice and increase your understanding (2 points each)

5. Complete all Lab tasks/exercises (10 points each)

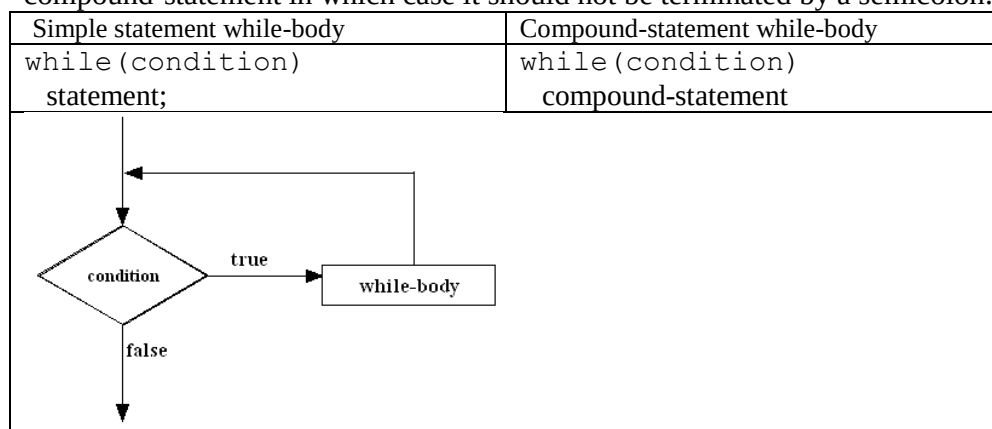
1. while-statement

A while-statement is used to execute a statement or a compound-statement zero or more times as long as the while-condition is true:

```
while (condition)
```

```
while-body
```

The while-body can be a simple statement in which case it must be terminated by a semicolon or it may be a compound-statement in which case it should not be terminated by a semicolon:



Examples : while-loop	output															
<pre>int n = 1; while(n <= 10){ System.out.printf("%d ", n); n += 2; }</pre>	1 3 5 7 9															
<pre>int k = 12; while(k > 6) System.out.printf("%d ", k--);</pre>	12 11 10 9 8 7															
<pre>int num = 2; System.out.println("number\tsquare root\tsquare\n"); while(num <= 5){ System.out. printf("%d\t%.2f\t%2d\n", num, Math.sqrt(num),num * num); num++; }</pre>	<table><tr><th>number</th><th>square root</th><th>square</th></tr><tr><td>2</td><td>1.41</td><td>4</td></tr><tr><td>3</td><td>1.73</td><td>9</td></tr><tr><td>4</td><td>2.00</td><td>16</td></tr><tr><td>5</td><td>2.24</td><td>25</td></tr></table>	number	square root	square	2	1.41	4	3	1.73	9	4	2.00	16	5	2.24	25
number	square root	square														
2	1.41	4														
3	1.73	9														
4	2.00	16														
5	2.24	25														

2. Sentinel controlled loops

In a program, a **sentinel** is a value that marks the end of a series of data values; but is not a data value itself. Sentinels may be used to control loops:

Example: Write a Java program fragment that prompts for and reads student grades in a quiz. It then calculates and displays the average. Use a negative value or a value greater than 100 as the sentinel.

```
int count = 0;
double grade, sumOfGrades = 0.0;
System.out.printf("Enter grade#%d (0 < or > 100 to terminate)\n", count+1);
grade = scanner.nextDouble();
while(grade >= 0 && grade <= 100){
    count++;
    sumOfGrades += grade;
    System.out.printf("Enter grade#%d (0 < or > 100 to terminate)\n", count+1);
    grade = scanner.nextDouble();
}
if(count == 0)
    System.out.println("Error: No valid grade entered");
else
    System.out.printf("Average = %.2f\n", sumOfGrades / count);
```

Example: Modify the above Java program fragment such that it also displays the maximum and minimum grade.

```
int count = 0;
double grade, sumOfGrades = 0.0;
System.out.printf("Enter grade#%d (0 < or > 100 to terminate)\n", count+1);
grade = scanner.nextDouble();
double max = grade; // Assume the first grade is the max
double min = grade; // Assume the first grade is the min
while(grade >= 0 && grade <= 100){
    count++;
    sumOfGrades += grade;

    if(grade > max)
        max = grade;
    else if(grade < min)
        min = grade;
    System.out.printf("Enter grade#%d (0 < or > 100 to terminate)\n", count+1);
    grade = scanner.nextDouble();
}
if(count == 0)
    System.out.println("Error: No valid grade entered");
else{
    System.out.printf("Average = %.2f\n", sumOfGrades / count);
    System.out.printf("Maximum = %.2f\n", max);
    System.out.printf("Minimum = %.2f\n", min);
}
```

Example: This program prints the average of salary values that are terminated with a sentinel.

```

import java.util.Scanner;
public class SentinelDemo{
    public static void main(String[] args) {
        double sum = 0;
        int count = 0;
        double salary = 0;
        System.out.print("Enter salaries, -1 to finish: ");
        Scanner in = new Scanner(System.in);

        // Process data until the sentinel is entered
        while (salary != -1) {
            salary = in.nextDouble();
            if (salary != -1) {
                sum = sum + salary;
                count++;
            }
        }

        // Compute and print the average
        if (count > 0) {
            double average = sum / count;
            System.out.println("Average salary: " + average);
        }
        else {
            System.out.println("No data");
        }
    }
}

```

Program Run

Enter salaries, -1 to finish: 10 10 40 -1
Average salary: 20

3. do-while statement

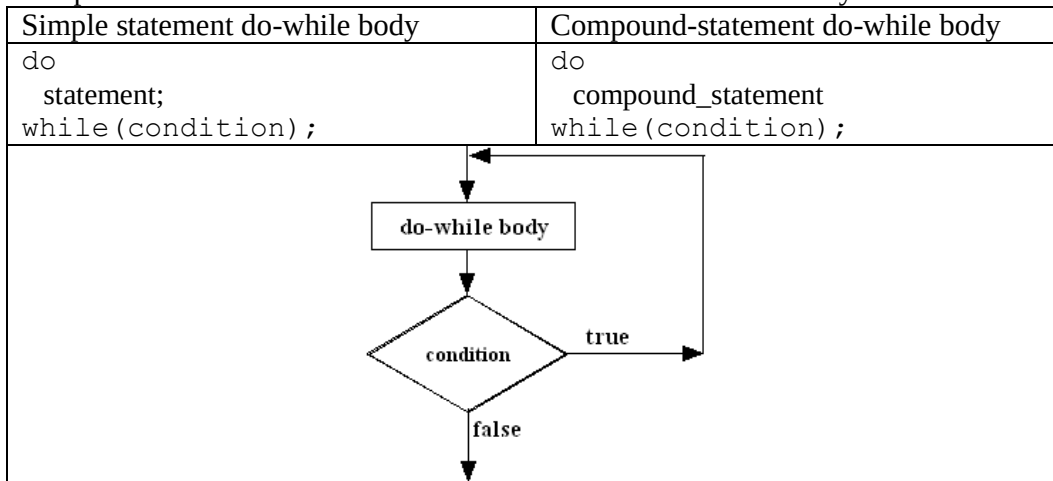
A do-while statement is used to execute a statement or a compound-statement one or more times as long as the do-while condition is true:

```

do
do-while body
while(condition);

```

The do-while body can be a simple statement in which case it must be terminated by a semicolon or it may be a compound-statement in which case it MUST NOT be terminated by a semicolon:



Examples: do-while loop	output
<pre> int n = 1; do{ System.out.printf("%d ", n); n += 2; }while(n <= 10); </pre>	1 3 5 7 9

<pre>int x = 25; do System.out.printf("%d ", x -= 5); while(x > 0);</pre>	<pre>20 15 10 5 0</pre>
--	-------------------------

Example: Write a Java program fragment that helps a child learn multiplication. The fragment displays two integers to be multiplied. The child is then given three chances to provide the correct answer.

```
int num1 = 7, num2 = 6, response, count = 0;
System.out.printf("What is %d * %d ?\n", num1, num2);
do{
    response = scanner.nextInt(); count++;
    if((response != num1 * num2) && (count < 3))
        System.out.println("Wrong response. Please try again");
} while((response != num1 * num2) && (count < 3 ));
if(response != num1 * num2)
    System.out.println("Sorry ! You were not successful");
else
    System.out.println("Congratulations ! You got the correct answer");
```

Example: A do-while loop can be used to validate input

```
int n;
do{
    System.out.println("Enter an integer number in the [10,100] interval: ");
    n = scanner.nextInt();
    if(n<10 || n>100)
        System.out.println("Sorry wrong input, try again");
}while (n < 10 || n > 100);
System.out.println("Now your input is correct");
```

Example: Using a do-while loop to recover from Scanner **java.util.InputMismatch Exception**

```
import java.util.*;
public class ErrorRecovery {
    public static void main(String[ ] args){
        int num = 0;
        boolean validInputType;
        Scanner scanner = new Scanner(System.in);
        do{
            System.out.println("Enter an int number in [10,100] range: ");
            try{
                num = scanner.nextInt( );
                validInputType = true;
            } //end try
            catch(InputMismatchException e){
                System.out.println("Error: " + e);
                validInputType = false;
                scanner.nextLine(); // remove new-line char from input
                buffer
            } //end catch
            if(validInputType){
                if(num < 10 || num >100)
                    System.out.println("Sorry input outside the range [10,
                    100], try again");
            } //end if
        }while (!validInputType || num < 10 || num > 100); //end do-while
        System.out.println("Now your input is correct");
    } //end main
} //end class
```

A do-while loop can be used in a menu driven program. The [example](#) shown below will continue running as long as the user did not enter the number 5.

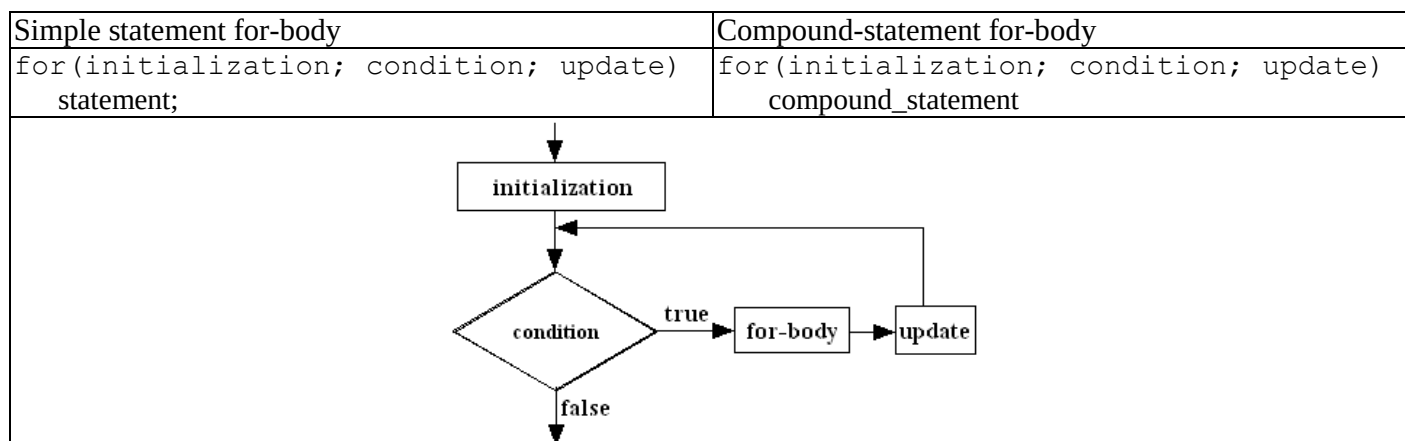
```
int choice;
do{
    System.out.println("1-Addition");
    System.out.println("2-Subtraction");
    System.out.println("3-Multiplication");
    System.out.println("4-Division");
    System.out.println("5-Exit");
    System.out.println("Enter your choice: ");
    choice = scanner.nextInt();
    // Here come the statements to do the different tasks
} while (choice != 5);
```

4. for-statement

A for-statement is usually used to repeat a statement or a compound-statement if the number of repetitions is known. The body of a for-loop may be executed zero or more times.

```
for(initialization; condition; update)
    for-body
```

The for-body can be a simple statement in which case it must be terminated by a semicolon, or it may be a compound-statement in which case it should not be terminated by a semicolon:



Examples: for-loop	output
<pre>int k; for(k = 7; k <= 12; k++) System.out.printf("%d ", k);</pre>	7 8 9 10 11 12
<pre>int x ; for(x = 8; x > 2; x--) if(x % 2 == 0) System.out.printf("%d is even\n", x); else System.out.printf("%d is odd\n", x);</pre>	8 is even 7 is odd 6 is even 5 is odd 4 is even 3 is odd
<pre>int counter = 1; int factorial = 1; for(counter = 1; counter <= 5; counter++) factorial = factorial * counter; System.out.printf("factorial of 5 is %d", factorial);</pre>	factorial of 5 is 120
<pre>// prints 6 characters per line char ch; for (ch='A'; ch<='Z'; ch++){ System.out.printf("%c\t",ch); if((ch-'A'+1)%6==0) System.out.println(); }</pre>	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

5. Equivalent loops

A loop can always be converted to an equivalent loop of a different type.

loop	equivalent loop
<pre>for(initialization;condition; update){ statement1; statement2; . . . statementN; }</pre>	<pre>initialization; while(condition) { statement1; statement2; . . . statementN; update; }</pre>
<pre>do{ statement1; statement2; . . . statementN; update; } while(condition);</pre>	<pre>statement1; statement2; . . . statementN; update; while(condition){ statement1; statement2; . . . statementN; update; }</pre>

Example: The following do-while loop: <pre>int n = 1; double x = 0, s; do{ s = 1.0 / (n * n); x = x + s; System.out.printf("x = %f\n", x); n++; } while(s > 0.01);</pre>	is equivalent to the while-loop: <pre>int n = 1; double x = 0, s; s = 1.0 / (n * n); x = x + s; System.out.printf("x = %f\n", x); n++; while(s > 0.01){ s = 1.0 / (n * n); x = x + s; System.out.printf("x = %f\n", x); n++; }</pre>
---	--

6. Nested loops

A loop statement may contain in its body one or more loop statements. There are two types of nested loops: independent and dependent. A nested loop is independent if its number of repetitions does not depend on an outer loop. A nested loop is dependent if its number of repetitions depends on an outer loop.

Examples: Nested loops	output
<pre>//Example of independent nested loop int m, n; for(m = 5; m >= 1; m--){ System.out.printf("m is now %d\n", m); for(n = 1; n <= 4; n++){ System.out.printf("n = %d ", n); System.out.println(); } }</pre>	<pre>m is now 5 n = 1 n = 2 n = 3 n = 4 m is now 4 n = 1 n = 2 n = 3 n = 4 m is now 3 n = 1 n = 2 n = 3 n = 4 m is now 2 n = 1 n = 2 n = 3 n = 4 m is now 1 n = 1 n = 2 n = 3 n = 4</pre>

```
//Example of dependent nested loop
int sum,i,j;
for(i=1;i<=10;i++){
    sum = 1;
    System.out.print("1");
    for(j=2; j<=i; j++){
        sum = sum + j;
        System.out.printf("+%d",j);
    }
    System.out.printf("=%d\n",sum);
}
```

```
1=1
1+2=3
1+2+3=6
1+2+3+4=10
1+2+3+4+5=15
1+2+3+4+5+6=21
1+2+3+4+5+6+7=28
1+2+3+4+5+6+7+8=36
1+2+3+4+5+6+7+8+9=45
1+2+3+4+5+6+7+8+9+10=55
```

Example: Write a Java program fragment that prompts for and reads three quiz grades for each student in a class of four students. The program then computes and displays the average for each student. Your program must be easily modifiable to handle any number of students and quizzes.

```
final int NUMSTUDENTS = 4, NUMQUIZES = 3;
double grade, studentTotal, studentAverage;
int m, n;
for(m = 1; m <= NUMSTUDENTS; m++) {
    studentTotal = 0.0;
    for(n = 1; n <= NUMQUIZES; n++) {
        System.out.printf("Enter QuizGrade#%d for student#%d\n", n, m);
        grade = scanner.nextDouble();
        studentTotal += grade;
    } //end for
    studentAverage = studentTotal / NUMQUIZES;
    System.out.printf("The average for student#%d is %.2f\n",studentAverage);
} //end for
```

Examples:

Nested Loops	Output	Explanation
<pre>for (i = 1; i <= 3; i++) { for (j = 1; j <= 4; j++) { Print "*" } System.out.println(); }</pre>	<pre>**** **** ****</pre>	Prints 3 rows of 4 asterisks each.
<pre>for (i = 1; i <= 4; i++){ for (j = 1; j <= 3; j++) { Print "*" } System.out.println(); }</pre>	<pre>*** *** *** ***</pre>	Prints 4 rows of 3 asterisks each.
<pre>for (i = 1; i <= 4; i++){ for (j = 1; j <= i; j++) { Print "*" } System.out.println(); }</pre>	<pre>* ** *** ****</pre>	Prints 4 rows of lengths 1, 2, 3, and 4.
<pre>for (i = 1; i <= 3; i++) { for (j = 1; j <= 5; j++){ if (j % 2 == 0) { Print "*" } else { Print "-" } } System.out.println(); }</pre>	<pre>-*-*- -*-*- -~-*-</pre>	Prints asterisks in even columns, dashes in odd columns.
<pre>for (i = 1; i <= 3; i++){ for (j = 1; j <= 5; j++){ if (i % 2 == j % 2) { Print "*" } else { Print " " } } System.out.println(); }</pre>	<pre>* * * * * * * *</pre>	Prints a checkerboard pattern.

Example: This program prints a table of powers of x.

```
public class PowerTable {
    public static void main(String[] args) {
        final int NMAX = 4;
        final double XMAX = 10;

        // Print table header
        for (int n = 1; n <= NMAX; n++){
            System.out.printf("%10d", n);
        }
        System.out.println();

        for (int n = 1; n <= NMAX; n++){
            System.out.printf("%10s", "x ");
        }
        System.out.println();

        // Print table body
        for (double x = 1; x <= XMAX; x++){
            // Print table row
            for (int n = 1; n <= NMAX; n++){
                System.out.printf("%10.0f", Math.pow(x, n));
            }
            System.out.println();
        }
    }
}
```

Program Run

1	2	3	4
x	x	x	x
1	1	1	1
2	4	8	16
3	9	27	81
4	16	64	256
5	25	125	625
6	36	216	1296
7	49	343	2401
8	64	512	4096
9	81	729	6561
10	100	1000	10000

7. Void Methods

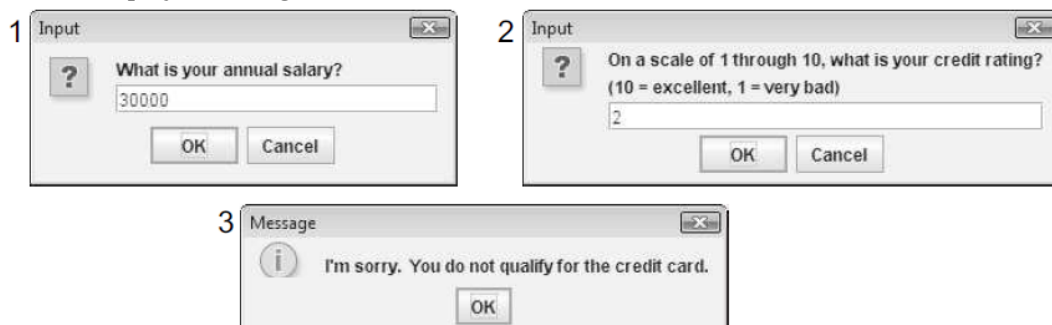
Method call statements may be used in control structures like loops, if statements, and switch statements.

Example: The LoopCall program places the displayMessage method call inside a loop. Notice where the displayMessage method is located and its structure and how it is used or called in the main method.

```
public class LoopCall{
    public static void main(String[] args){
        System.out.println("Hello from the main method.");
        for (int i = 0; i < 5; i++){
            displayMessage();
        }
        System.out.println("Back in the main method.");
    }

    /**
     * The displayMessage method displays a greeting.
     */
    public static void displayMessage(){
        System.out.println("Hello from the displayMessage method.");
    }
}
```

Example: The **CreditCard** program shows another example. It asks the user to enter his or her annual salary and credit rating. The program then determines whether the user qualifies for a credit card. One of two void methods, **qualify** or **noQualify**, is called to display a message.




```

import javax.swing.JOptionPane;
/**
    This program uses two void methods.
*/
public class CreditCard{
    public static void main(String[] args){
        double salary;        // Annual salary
        int creditRating;      // Credit rating
        String input;          // To hold the user's input

        // Get the user's annual salary.
        input = JOptionPane.showInputDialog("What is " +
                                           "your annual salary?");
        salary = Double.parseDouble(input);

        // Get the user's credit rating (1 through 10).
        input = JOptionPane.showInputDialog("On a scale of " +
                                           "1 through 10, what is your credit rating?\n" +
                                           "(10 = excellent, 1 = very bad)");
        creditRating = Integer.parseInt(input);

        // Determine whether the user qualifies.
        if (salary >= 20000 && creditRating >= 7)
            qualify();
        else
            noQualify();

        System.exit(0);
    }
}

/**
    The qualify method informs the user that he
    or she qualifies for the credit card.
*/
public static void qualify(){
    JOptionPane.showMessageDialog(null, "Congratulations! " +
                                     "You qualify for the credit card!");
}

/**
    The noQualify method informs the user that he
    or she does not qualify for the credit card.
*/
public static void noQualify(){
    JOptionPane.showMessageDialog(null, "I'm sorry. You " +
                                     "do not qualify for the credit card.");
}
}

```

Example: **DeepAndDeeper** demonstrates hierarchical method calls. Follow the call of the method from the start of the program, the **main** method to **deep**, then from **deep** to **deeper**.

```

public class DeepAndDeeper {
    public static void main(String[] args){
        System.out.println("I am starting in main.");
        deep();
        System.out.println("Now I am back in main.");
    }

    /**
        The deep method displays a message and then calls
        the deeper method.
    */
    public static void deep() {
        System.out.println("I am now in deep.");
        deeper();
        System.out.println("Now I am back in deep.");
    }

    /**
        The deeper method simply displays a message.
    */
    public static void deeper() {
        System.out.println("I am now in deeper.");
    }
}

```

Value Passing

A method may be written so it accepts arguments. Data can then be passed into the method when it is called. By using parameter variables, you can design your own methods that accept data this way. A *parameter variable*, sometimes simply referred to as a *parameter*, is a special variable that holds a value being passed into a method.

Example: `PassArg` demonstrates a method with a parameter. Inside the `displayValue` method, the variable `num` will contain the value of whatever argument was passed into it.

```
public class PassArg {
    public static void main(String[] args) {
        int x = 10;

        System.out.println("I am passing values to displayValue.");
        displayValue(5);           // Pass 5
        displayValue(x);           // Pass 10
        displayValue(x * 4);       // Pass 40
        displayValue(Integer.parseInt("700")); // Pass 700
        System.out.println("Now I am back in main.");
    }

    /**
     * The displayValue method displays the value
     * of its integer parameter.
     */
    public static void displayValue(int num) {
        System.out.println("The value is " + num);
    }
}
```

Notice that you declare the data type of the passing parameter in the method `displayValue` but when calling the method in `main`, you pass value or variable but not the datatype. When calling the method, the following statement will cause an error:

```
displayValue(int x); // Error!
```

The method call should appear as follows:

```
displayValue(x); // Correct
```

Arguments Are Passed by Value

In Java, all arguments of the primitive data types are *passed by value*, which means that only a copy of an argument's value is passed into a parameter variable. A method's parameter variables are separate and distinct from the arguments that are listed inside the parentheses of a method call. If a parameter variable is changed inside a method, it has no effect on the original argument.

Example: This `PassByValue` program demonstrates that only a copy of an argument is passed into a method.

```

public class PassByValue{
    public static void main(String[] args) {
        int number = 99; // number starts with 99

        // Display the value in number.
        System.out.println("number is " + number);

        // Call changeMe, passing the value in number
        // as an argument.
        changeMe(number);

        // Display the value in number again.
        System.out.println("number is " + number);
    }

    /**
     The changeMe method accepts an argument and then
     changes the value of the parameter.
    */
    public static void changeMe(int myValue) {
        System.out.println("I am changing the value.");
        // Change the myValue parameter variable to 0.
        myValue = 0;

        // Display the value in myValue.
        System.out.println("Now the value is " + myValue);
    }
}

```

Passing Multiple Arguments

Often it is useful to pass more than one argument to a method. Here is an [example](#) method that accepts two arguments:

```

public static void showSum(double num1, double num2)
{
    double sum;    // To hold the sum

    sum = num1 + num2;
    System.out.println("The sum is " + sum);
}

```

Notice that two parameter variables, num1 and num2, are declared inside the parentheses in the method header. This is often referred to as a *parameter list*. Also notice that a comma separates the declarations. Here is an example of a statement that calls the method:

```
showSum(5, 10);
```

This statement **passes the arguments** 5 and 10 into the method. The arguments are passed into the parameter variables in the order that they appear in the method call. In other words, the first argument is passed into the first parameter variable, the second argument is passed into the second parameter variable, and so forth. So, this statement causes 5 to be passed into the num1 parameter and 10 to be passed into the num2 parameter.

Passing Object References to a Method

So far you've seen examples of methods that accept primitive values as arguments. You can also write methods that accept references to objects as arguments. For [example](#), look at the following method:

```

public static void showLength(String str) {
    System.out.println(str + " is " + str.length() +
        " characters long.");
}

```

This method accepts a String object reference as its argument, and displays a message showing the number of characters in the object. The following code shows an example of how to call the method:

```
String name = "Warren";
showLength(name);
```

Output: Warren is 6 characters long.

Local Variables

A local variable is **declared inside a method** and is not accessible to statements outside the method. Different methods can have local variables with the same names because the methods cannot see each other's local variables.

This [example](#) program demonstrates that two methods may have local variables with the same name.

```
public class LocalVars {
    public static void main(String[] args) {
        texas();
        california();
    }

    /* The texas method has a local variable named birds.
    */
    public static void texas() {
        int birds = 5000;
        System.out.println("In texas there are " +
            birds + " birds.");
    }

    /* The california method also has a local variable named birds.
    */
    public static void california() {
        int birds = 3500;
        System.out.println("In california there are " +
            birds + " birds.");
    }
}
```

8. Value Methods

When you are writing a value-returning method, you must decide what type of value the method will return. This is because you must specify the data type of the return value in the method header. Recall that a void method, which does not return a value, uses the key word void as its return type in the method header. A value-returning method will use int, double, boolean, or any other valid data type in its header. Here is an [example](#) of a method that returns an int value:

Return type

```
public static int sum(int num1, int num2) {
    int result;
    result = num1 + num2;
    return result;
}
```

Which can be shorted in this manner:

```
public static int sum(int num1, int num2) {
    return num1 + num2;
}
```

Putting this example altogether into a class. Try out [example](#) ValueReturn.

```
public class ValueReturn {
    public static void main(String[] args) {
        int total, value1 = 20, value2 = 40;

        // Call the sum method, passing the contents of
        // value1 and value2 as arguments. Assign the
        // return value to the total variable.
        total = sum(value1, value2);

        // Display the contents of all these variables.
        System.out.println("The sum of " + value1 +
            " and " + value2 + " is " +
            total);
    }

    /* The sum method returns the sum of its two parameters.
    @param num1 The first number to be added.
    @param num2 The second number to be added.
    @return The sum of num1 and num2. */
    public static int sum(int num1, int num2){
        int result; // result is a local variable

        // Assign the value of num1 + num2 to result.
        result = num1 + num2;

        // Return the value in the result variable.
        return result;
    }
}
```

Example: Some ingredients found in recipe books are measured in cups but at the grocery store these ingredients are sold only by ounce. Write a simple program to convert cups to fluid ounces. 1 cup is equivalent to 8 fluid ounces. You design the following algorithm:

1. Get the number of cups from the user.
2. Convert the number of cups to fluid ounces.
3. Display the result.

Based on the design above, the class will have the following methods:

- **getCups**—This method will prompt the user to enter the number of cups, and then return that value as a double.
- **cupsToOunces**—This method will accept the number of cups as an argument and then return the equivalent number of fluid ounces as a double.
- **displayResults**—This method displays a message indicating the results of the conversion.

<pre>import javax.swing.JOptionPane; public class CupConverter { public static void main(String[] args) { double cups; // To hold the number of cups double ounces; // To hold the number of ounces // Get the number of cups. cups = getCups(); // Convert the cups to fluid ounces. ounces = cupsToOunces(cups); // Display the results. displayResults(cups, ounces); System.exit(0); } /* The getCups method prompts the user to enter a number of cups. @return The number of cups entered by the user. */ public static double getCups() { String input; // To hold input. double numCups; // To hold cups. // Get the number of cups from the user. input = JOptionPane.showInputDialog("This program converts measurements\n" + "in cups to fluid ounces. For your\n" + "reference the formula is:\n" + " 1 cup = 8 fluid ounces\n\n" + "Enter the number of cups."); } }</pre>	<pre>// Convert the input to a double. numCups = Double.parseDouble(input); // Return the number of cups. return numCups; } /* The cupsToOunces method converts a number of cups to fluid ounces, using the formula 1 cup = 8 fluid ounces. @param numCups The number of cups to convert. @return The number of ounces. */ public static double cupsToOunces(double numCups) { return numCups * 8.0; } /* The displayResults method displays a message showing the results of the conversion. @param cups A number of cups. @param ounces A number of ounces. */ public static void displayResults(double cups, double ounces){ // Display the number of ounces. JOptionPane.showMessageDialog(null, cups + " cups equals " + ounces + " fluid ounces."); } }</pre>
--	--

Returning a boolean Value

Frequently there is a need for a method that tests an argument and returns a true or false value indicating whether or not a condition exists. Such a method would return a boolean value. For example, the following method accepts an argument and returns true if the argument is within the range of 1 through 100, or false otherwise:

```
public static boolean isValid(int number){
    boolean status;

    if (number >= 1 && number <= 100)
        status = true;
    else
        status = false;
    return status;
}
```

The following code shows an if-else statement that uses a call to the method:

```
int value = 20;
if (isValid(value))
    System.out.println("The value is within range.");
else
    System.out.println("The value is out of range.");
```

When this code executes, the message “The value is within range.” will be displayed.

Problem Solving with Methods

A large, complex problem can be solved a piece at a time by methods. We introduced the idea of using methods to “divide and conquer” a problem. Often the best way to solve a complex problem is to break it down into smaller problems, and then solve the smaller problems. The process of breaking down a problem into smaller pieces is called *functional decomposition*. In functional decomposition, instead of writing one long method that contains all of the statements necessary to solve a problem, small methods are written, which each solve a specific part of the problem. These small methods can then be executed in the desired order to solve the problem.

Example: The **SalesReport** program reads 30 days of sales amounts from a file (*MonthlySales.txt* is provided), and then displays the total sales and average daily sales. Keep the file in the same folder as your .java file. Here’s the algorithm in pseudocode:

Ask the user to enter the name of the file.

Get the total of the sales amounts in the file.

Calculate the average daily sales.

Display the total and average daily sales.

```
import java.util.Scanner; // For Scanner class
import java.io.*; // For file I/O classes
import javax.swing.JOptionPane; // For JOptionPane class

public class SalesReport {
    public static void main(String[] args)
        throws IOException {
        final int NUM_DAYS = 30; // Number of days of sales
        String filename; // The name of the file to open
        double totalSales; // Total sales for period
        double averageSales; // Average daily sales

        // Get the name of the file.
        filename = getFileName();

        // Get the total sales from the file.
        totalSales = getTotalSales(filename);

        // Calculate the average.
        averageSales = totalSales / NUM_DAYS;

        // Display the total and average.
        displayResults(totalSales, averageSales);
        System.exit(0);
    }

    /* The getFileName method prompts the user to enter
       the name of the file to open.
       @return A reference to a String object containing
       the name of the file. */
    public static String getFileName() {
        String file; // To hold the file name

        // Prompt the user to enter a file name.
        file = JOptionPane.showInputDialog(
            "Enter the name of the file\n" +
            "containing 30 days of sales amounts.");

        // Return the name.
        return file;
    }

    /* The getTotalSales method opens a file and
       reads the daily sales amounts, accumulating
       the total. The total is returned.
       @param filename The name of the file to open.
       @return The total of the sales amounts. */
    public static double getTotalSales(String filename)
        throws IOException {
        double total = 0.0; // Accumulator
        double sales; // A daily sales amount

        // Open the file.
        File file = new File(filename);
        Scanner inputFile = new Scanner(file);

        // This loop processes the lines read from the file,
        // until the end of the file is encountered.
        while (inputFile.hasNext()) {
            // Read a double from the file.
            sales = inputFile.nextDouble();

            // Add sales to the value already in total.
            total += sales;
        }

        // Close the file.
        inputFile.close();

        // Return the total sales.
        return total;
    }

    /* The displayResults method displays the total and
       average daily sales.
       @param total The total sales.
       @param avg The average daily sales. */
    public static void displayResults(double total, double avg) {
        // Display the formatted total and average sales.
        JOptionPane.showMessageDialog(null,
            String.format("The total sales for the period is $%,.2f\n" +
                "The average daily sales were $%,.2f",
                    total, avg));
    }
}
```

9. Lab Tasks /Exercises

Each task/exercise is worth 10 marks.

Task01: Using a **while** sentinel controlled loop, write a program that reads a radius from the user and displays the area and circumference of the circle with that radius. This task is repeated until the user types a negative value. Assume that the radius is in centimeters. [$Area = \pi r^2$, $C = 2 \pi r$] Sample run:


```

Enter radius (0 to quit): 3.5
radius = 3.50 cm, area = 38.50 square cm, circumference = 22.00 cm
Enter radius (0 to quit): 5
radius = 5.00 cm, area = 78.57 square cm, circumference = 31.43 cm
Enter radius (0 to quit): 10
radius = 10.00 cm, area = 314.29 square cm, circumference = 62.86 cm
Enter radius (0 to quit): 23.46
radius = 23.46 cm, area = 1729.76 square cm, circumference = 147.46 cm
Enter radius (0 to quit): 0

```

Task02: Using **nested for loops**, write a Java program to print the pattern shown in the sample runs below on the screen. Sample runs:

<pre> Enter an integer from 1 to 10: 8 x xx xxx xxxx xxxxx xxxxxx xxxxxxx xxxxxxxx xxxxxxxxx xxxxxxxxxx </pre>	<pre> Enter an integer from 1 to 10: 3 x xx xxx </pre>
--	--

Task03: Using a **do-while** loop, write a Java program to check that the user has typed a letter. The program must not stop until the user types a lowercase or uppercase letter. Sample runs:

<pre> Enter a lower case or uppercase letter: 4 Sorry you did not type a letter. Try again Enter a lower case or uppercase letter: % Sorry you did not type a letter. Try again Enter a lower case or uppercase letter: # Sorry you did not type a letter. Try again Enter a lower case or uppercase letter: A You typed an uppercase letter </pre>	<pre> Enter a lower case or uppercase letter: * Sorry you did not type a letter. Try again Enter a lower case or uppercase letter: \ Sorry you did not type a letter. Try again Enter a lower case or uppercase letter: \$ Sorry you did not type a letter. Try again Enter a lower case or uppercase letter: d You typed a lowercase letter </pre>
---	---

Task04: Write a program that prints all powers of 2 from 2^0 up to 2^{20}

Task05: A simple random generator is obtained by the formula

$$r_{\text{new}} = (a * r_{\text{old}} + b) \% m$$

and then setting r_{old} to r_{new} . If m is chosen as 2^{32} , then you can compute

$$r_{\text{new}} = a * r_{\text{old}} + b$$

because the truncation of an overflowing result to the int type is equivalent to computing the remainder.

Write a program that asks the user to enter a seed value for r_{old} . (Such a value is often called a *seed*). Then print the first 100 random integers generated by this formula, using $a = 32310901$ and $b = 1729$.

Task06: In a predator-prey simulation, you compute the populations of predators and prey, using the following equations:

$$prey_{n+1} = prey_n \times (1 + A - B \times pred_n)$$

$$pred_{n+1} = pred_n \times (1 - C + D \times prey_n)$$

Here, A is the rate at which prey birth exceeds natural death, B is the rate of predation, C is the rate at which predator deaths exceed births without food, and D represents predator increase in the presence of food.

Write a program that prompts users for these rates, the initial population sizes, and the number of periods.

Then print the populations for the given number of periods. As inputs, try $A = 0.1$, $B = C = 0.01$, and $D = 0.00002$ with initial prey and predator populations of 1,000 and 20.

Task07: Write a program that will ask the user to enter the width and length of a rectangle, and then display the rectangle's area. The program calls the following methods, which have not been written:

- `getLength`—This method should ask the user to enter the rectangle's length, and then return that value as a double.
- `getWidth`—This method should ask the user to enter the rectangle's width, and then return that value as a double.
- `getArea`—This method should accept the rectangle's length and width as arguments, and return the rectangle's area. The area is calculated by multiplying the length by the width.

• displayData—This method should accept the rectangle’s length, width, and area as arguments, and display them in an appropriate message on the screen.

Task08: A painting company has determined that for every 115 square feet of wall space, one gallon of paint and eight hours of labor will be required. The company charges \$18.00 per hour for labor. Write a program that allows the user to enter the number of rooms to be painted and the price of the paint per gallon. It should also ask for the square feet of wall space in each room. The program should have methods that return the following data:

- The number of gallons of paint required
 - The hours of labor required
 - The cost of the paint
 - The labor charges
 - The total cost of the paint job
- Then it should display the data on the screen.

Task09: When an object is falling due to gravity, the following formula can be used to determine the distance the object falls in a specific time period: $d = \frac{1}{2}gt^2$. The variables in the formula are as follows: d is the distance in meters, g is 9.8, and t is the amount of time, in seconds, that the object has been falling. Write a method named **fallingDistance** that accepts an object’s falling time (in seconds) as an argument. The method should return the distance, in meters, that the object has fallen during that time interval. Demonstrate the method by calling it in a loop that passes the values 1 through 10 as arguments, and displays the return value.

Task10: Write a program that asks the user to enter a distance in meters. The program will then present the following menu of selections:

1. Convert to kilometers
2. Convert to inches
3. Convert to feet
4. Quit the program

The program will convert the distance to kilometers, inches, or feet, depending on the user’s selection. Here are the specific requirements:

1. Write a void method named **showKilometers**, which accepts the number of meters as an argument. The method should display the argument converted to kilometers. Convert the meters to kilometers using the following formula: $kilometers = meters * 0.001$
2. Write a void method named **showInches**, which accepts the number of meters as an argument. The method should display the argument converted to inches. Convert the meters to inches using the following formula: $inches = meters * 39.37$
3. Write a void method named **showFeet**, which accepts the number of meters as an argument. The method should display the argument converted to feet. Convert the meters to feet using the following formula: $feet = meters * 3.281$
4. Write a void method named menu that displays the menu of selections. This method should not accept any arguments.
5. The program should continue to display the menu until the user enters 4 to quit the program.
6. The program should not accept negative numbers for the distance in meters.
7. If the user selects an invalid choice from the menu, the program should display an error message.

Here is an example session with the program, using console input. The user’s input is shown in bold.

Enter a distance in meters: 500 [Enter] 1. Convert to kilometers 2. Convert to inches 3. Convert to feet 4. Quit the program	Enter your choice: 3 [Enter] 500 meters is 1640.5 feet. 1. Convert to kilometers 2. Convert to inches 3. Convert to feet 4. Quit the program
Enter your choice: 1 [Enter] 500 meters is 0.5 kilometers. 1. Convert to kilometers 2. Convert to inches 3. Convert to feet 4. Quit the program	Enter your choice: 4 [Enter] Bye!

Task11: The profit from the sale of a stock can be calculated as follows: $Profit = ((NS \times SP) - SC) - ((NS \times PP) + PC)$ where NS is the number of shares, PP is the purchase price per share, PC is the purchase commission paid, SP is the sale price per share, and SC is the sale commission paid. If the calculation yields a positive value, then the sale of the stock resulted in a profit. If the calculation yields a negative number, then the sale resulted in a loss. Write a method that accepts as arguments the number of shares, the purchase price per share, the purchase commission paid, the sale price per share, and the sale commission paid. The method should return the profit (or loss) from the sale of stock. Demonstrate the method in a program that asks the user to enter the necessary data and displays the amount of the profit or loss.

Task12: Write **isPrime** method and use this method to print all the prime numbers from 1 through 100. Tip: Recall that the % operator divides one number by another, and returns the remainder of the division. In an expression such as num1 % num2, the % operator will return 0 if num1 is evenly divisible by num2.